

Deep Bluffing: A Monte-Carlo Equity and Game-Theoretic Expected-Value Agent for Heads-Up Limit Hold'em

ICG Summer Project 2026–27

July 19, 2026

Abstract

This report documents the design and mathematical justification of an autonomous agent for Heads-Up Limit Texas Hold'em (HULHE), built for the Deep Bluffing capstone. The agent's strategy archetype is a *Monte-Carlo equity estimator* combined with a *closed-form expected-value decision rule*, deliberately balanced against exploitation using a bluffing frequency derived from simplified poker game theory. We justify the state representation, the statistical properties of the equity estimator, the EV decision boundary, the bluffing-frequency formula, and the performance engineering required to make the approach tractable across a 10,000-hand tournament. Empirical results against a battery of baseline agents are reported.

Contents

1	Problem Setting	2
2	State Representation	2
2.1	Card Encoding	2
3	Monte-Carlo Hand Equity Estimation	2
3.1	Definition	2
3.2	Estimator and Its Error	3
4	Expected-Value Decision Rule	3
4.1	Break-Even Equity (Pot Odds)	3
4.2	Three-Zone Decision Boundary	3
4.3	Balanced Bluffing via Minimum Defense Frequency	4
5	Hand Evaluation	4
6	Performance Engineering for Tournament Scale	4
7	Empirical Validation	5
7.1	Correctness	5
7.2	Exploitability Sanity Checks	5
8	Limitations and Future Work	5
9	Conclusion	6

1 Problem Setting

Heads-Up Limit Hold'em is a two-player, imperfect-information, sequential game with a fixed four-street structure (pre-flop, flop, turn, river) and a discrete, bounded action set at every decision point: {FOLD, CALL, RAISE}. The fixed-limit structure and the 4-bet cap per street bound the size of the game tree per street, but the hidden information – the opponent's two hole cards, drawn from $\binom{50}{2} = 1225$ possibilities from the agent's point of view – makes exact solution (e.g. full-game CFR to a tight ε) computationally heavy to do well within a summer-project timeframe. We therefore adopt an architecture that is mathematically transparent, cheap to compute at scale (the arena plays 10,000 hands per match), and provably not maximally exploitable, rather than an exact-solution approach.

2 State Representation

At every call to `get_action`, the agent is given (`hole_cards`, `community_cards`, `pot_size`, `stack_size`, `amount_to_call`, `leg`). We deliberately do *not* hand-craft a large numeric feature vector (e.g. one-hot rank/suit encodings) because our decision rule does not consume features directly – it consumes a single derived scalar, **hand equity** $e \in [0, 1]$, estimated by Monte-Carlo rollout (Section 3). This is a considerable dimensionality reduction: the entire imperfect-information state (hole, board), which for hole+board sizes ranging from 2 to 7 cards has $\binom{52}{2} \cdot \binom{50}{k}$ possible realisations depending on the street, is collapsed to one calibrated number that is directly commensurable with the other scalar in the decision, the pot-odds break-even threshold p^* (Section 4). This keeps the strategy interpretable and lets us state the decision rule as literally two numbers being compared.

2.1 Card Encoding

Cards are represented internally as (`value`, `suit`) pairs with `value` $\in \{2, \dots, 14\}$ (Ace high = 14), via the mapping implemented in `card.py`. A dedicated `Card` class provides `__lt__`/`__eq__` so that sorting and comparing hands during evaluator development and testing is natural Python, while the hot Monte-Carlo loop in `agent.py` operates on raw (`int`, `str`) tuples to avoid object-construction overhead (Section 6).

3 Monte-Carlo Hand Equity Estimation

3.1 Definition

Given the agent's hole cards H and the currently revealed board B ($|B| \in \{0, 3, 4, 5\}$), define equity as

$$e(H, B) = \Pr[\text{agent wins} \mid H, B] + \frac{1}{2} \Pr[\text{tie} \mid H, B],$$

where the probability is taken over a uniformly random opponent hand drawn from the remaining unseen cards and a uniformly random completion of the board to five cards. This is exactly the win probability the agent would have in a hypothetical all-in showdown from the current state, ignoring future betting – a standard and mathematically clean proxy for "how good is my hand right now."

3.2 Estimator and Its Error

We estimate e by rollout: draw N i.i.d. samples of (opponent hole cards, remaining board cards) without replacement from the known-unseen deck, evaluate the winner of each sampled showdown with the from-scratch hand evaluator (Section 5), and average:

$$\hat{e} = \frac{1}{N} \sum_{i=1}^N X_i, \quad X_i = \begin{cases} 1 & \text{agent wins sample } i \\ 0.5 & \text{tie} \\ 0 & \text{opponent wins} \end{cases}.$$

Since the X_i are i.i.d. with mean e and variance $\sigma^2 = e(1 - e)$ (treating ties' 0.5 contribution as approximately binomial for this bound), the Law of Large Numbers guarantees $\hat{e} \rightarrow e$ almost surely, and the standard error of the estimate is

$$\text{SE}(\hat{e}) = \sqrt{\frac{e(1 - e)}{N}}.$$

At the worst case $e = 0.5$, $N = 80$ (our post-flop rollout count) gives $\text{SE} \approx \sqrt{0.25/80} \approx 0.056$, i.e. roughly a $\pm 11\%$ 95% confidence half-width on the equity estimate. This is precise enough to separate clear value bets / folds from the close-to-indifferent zone, which is exactly where the margins `VALUE_RAISE_MARGIN` = 0.06 and `FOLD_MARGIN` = 0.04 in `agent.py` are calibrated relative to the estimator noise: they are of the same order as $\text{SE}(\hat{e})$, so the agent does not flip decisions on rollout noise alone, while still reacting to genuine equity swings across streets. Pre-flop uses $N = 60$ (more streets remain, so precision matters slightly less relative to future information) and post-flop uses $N = 80$ (chosen to keep total tournament runtime tractable – see Section 6 – while keeping SE in the same regime).

4 Expected-Value Decision Rule

4.1 Break-Even Equity (Pot Odds)

Given `pot_size` = P and `amount_to_call` = c , calling risks c to win the current pot P (plus, implicitly, the c itself back on a win). The call is profitable in expectation iff

$$e \cdot P - (1 - e) \cdot c > 0 \iff e > \underbrace{\frac{c}{P + c}}_{p^*}.$$

p^* is the classical *pot-odds break-even threshold*. The agent computes it directly from the state it is handed – no lookup table required.

4.2 Three-Zone Decision Boundary

We partition e relative to p^* (or relative to 0.5 when $c = 0$, i.e. the agent is closing the action with a check available) into three zones:

- **Value zone** ($e \geq p^* + \delta_v$): raise for value.
- **Indifference zone** ($p^* - \delta_f \leq e < p^* + \delta_v$): call – the pot odds justify continuing but the edge is not large enough to raise profitably against a rational opponent's likely continuing range.
- **Fold zone** ($e < p^* - \delta_f$): fold – *unless* a balanced bluff is triggered (Section 4.3).

with $\delta_v = \text{VALUE_RAISE_MARGIN} = 0.06$ and $\delta_f = \text{FOLD_MARGIN} = 0.04$, both chosen to sit just above the Monte-Carlo estimator’s standard error (Section 3.2) so decisions are driven by genuine equity signal, not estimator noise.

4.3 Balanced Bluffing via Minimum Defense Frequency

A decision rule that only ever raises with $e \geq p^* + \delta_v$ and only ever calls/folds otherwise is trivially exploitable: a defending opponent could fold every hand that does not itself clear a very high equity threshold, safe in the knowledge that the agent never bets without the goods. This motivates the classic game-theoretic countermeasure. Consider a simplified one-street model: the agent holds a hand that is a guaranteed loser at showdown but bets an amount c into pot P ; a rational opponent, facing this single possible bluff-or-value bet with some population mix, is indifferent between calling and folding exactly when the bettor’s bluffing frequency f satisfies

$$f = \frac{c}{P + 2c},$$

which is the same ratio that (from the opponent’s perspective) defines their *Minimum Defense Frequency* against being exploited by an always-bluff strategy. We use this ratio directly as our target bluff frequency when the agent is in the fold zone but a raise is legal, scaled by a constant $\text{BLUFF_SCALE} = 0.9 < 1$: since real opponents (including simple baseline bots) are not perfectly equilibrium-calling, a slightly *lower* bluff frequency than the theoretical indifference point is more robust and less exploitable in practice, at the cost of a small, bounded amount of theoretical EV against a perfect Nash opponent. When it is the agent’s own action to open, i.e. $c = 0$ (checked to), we use a flat, empirically tuned semi-bluff frequency of $0.9 \times 0.15 = 13.5\%$ on weak hands ($e < 0.35$) instead, since the pot-odds-indifference derivation above requires a nonzero bet already on the table.

5 Hand Evaluation

Two independent implementations of 7-card hand ranking are provided in `evaluator.py`:

1. **Reference evaluator** (`best_hand_rank`): enumerates all $\binom{7}{5} = 21$ five-card sub-hands via `itertools.combinations` and returns the maximum rank tuple. Simple and obviously correct; used once per hand at showdown, where its $O(21)$ cost is immaterial.
2. **Fast evaluator** (`fast_rank7`): a single $O(1)$ -ish pass (rank/suit counting via `collections.Counter`, one straight/flush scan) that computes the identical rank tuple without enumerating combinations. Used inside the Monte-Carlo rollout hot loop, which calls the evaluator on the order of $2 \times (\text{rollouts}) \times (\text{decisions per hand})$ times.

Both a full 5-card ranking function (straight flush through high card, with correct kicker handling and the wheel straight `A2345`) and the fast 7-card variant are exercised by an automated, randomised equivalence test (`test_engine.py::test_evaluator_equivalence`), which compares their output on 200,000+ random 7-card hands with zero mismatches during development.

6 Performance Engineering for Tournament Scale

A 10,000-hand match, each hand involving up to four betting streets with up to four bet actions each (the cap rule), can call `get_action` roughly 6–10 times per hand. A naive implementation – constructing `Card` objects and enumerating all $\binom{7}{5}$ combinations inside every rollout sample –

measured at approximately 108 ms per hand, i.e. roughly 18 minutes for 10,000 hands against a single opponent, which does not scale to a round-robin tournament against many peers. Two optimisations reduced this to approximately 9.6 ms per hand (an $\sim 11\times$ speed-up):

1. Switching the rollout’s internal evaluator calls from the combinations-based reference evaluator to `fast_rank7` (Section 5), and operating on raw (`value`, `suit`) tuples rather than `Card` objects during rollouts.
2. **Equity caching within a street.** A raise/re-raise exchange on a single street calls `get_action` multiple times with *identical* hole and board cards. Since equity only depends on these two quantities, the agent caches $\hat{e}(H, B)$ keyed on `(sorted(hole), sorted(board))` and invalidates the cache exactly when a new hand begins (detected by a change in hole cards), avoiding redundant rollouts during bet-capped raising wars.

7 Empirical Validation

7.1 Correctness

`test_engine.py` verifies: (a) evaluator equivalence (above); (b) 1000+ simulated hands complete with no exceptions and all payoffs are exactly zero-sum; (c) the engine’s safety wrapper (`_safe_get_action`) never had to override `CustomPokerBot`’s chosen action during any test run, i.e. the bot always returns a legal action.

7.2 Exploitability Sanity Checks

Table 1 reports `bb/100` (big blinds won per 100 hands, alternating the dealer button every hand) for `CustomPokerBot` against a battery of simple baseline opponents implemented in `baselines.py`, over 2000-hand matches.

Opponent	bb/100
AlwaysFold	+73.98
AlwaysCall	+107.68
AlwaysRaise (maniac)	+173.83
Random (uniform over legal actions)	+230.58
TightAggressive (crude premium-hands heuristic)	+44.80

Table 1: `CustomPokerBot`’s win rate against baseline opponents (2000 hands each, single seed).

The agent is solidly profitable against every baseline tested, including the crude heuristic `TightAggressiveBot`, and is most profitable against the unconditionally aggressive and purely random opponents – exactly the qualitative ordering one would expect from an equity-driven EV bot, since both of those baselines systematically overplay weak holdings that our agent’s pot-odds comparison correctly punishes.

8 Limitations and Future Work

The current agent ignores multi-street planning: equity is computed myopically from the current state, without modelling how the opponent’s strategy might change across future streets (no explicit game tree search or belief update on the opponent’s range beyond “uniformly random”). A natural extension is range-based equity (weighting the opponent-hand sampler by a plausible continuing

range conditioned on their action history) or a lightweight Counterfactual Regret Minimization solve of the abstracted, bet-capped game tree, which would converge to a Nash equilibrium strategy rather than the current best-response-flavoured heuristic. A DQN-based approach (as used in the CartPole prerequisite assignment) was considered but de-prioritised for this heads-up limit setting because the imperfect-information, opponent-dependent reward structure requires either self-play or a population of opponents to train against meaningfully, which was judged to be a larger undertaking than the summer-project timeframe allowed without risking an unstable or under-trained policy at submission time.

9 Conclusion

The submitted agent combines a statistically-grounded equity estimator, a closed-form EV decision boundary calibrated against the estimator’s own noise, and a game-theoretically motivated bluffing frequency, all implemented without external poker libraries or pretrained lookup tables, and engineered to run a 10,000-hand tournament in under two minutes. Empirically it is profitable against every baseline tested.